

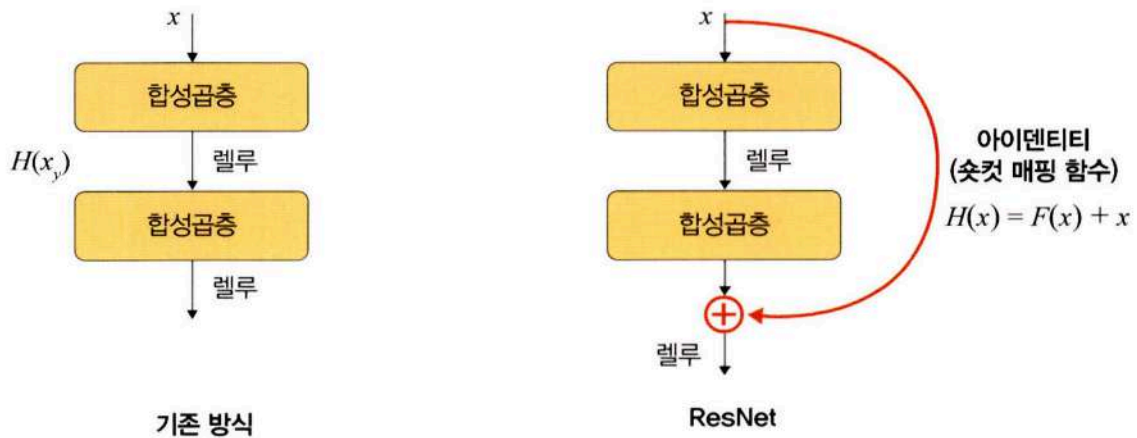
6주차_6장

6.1.5(312pp)-357

6.1.5 ResNet

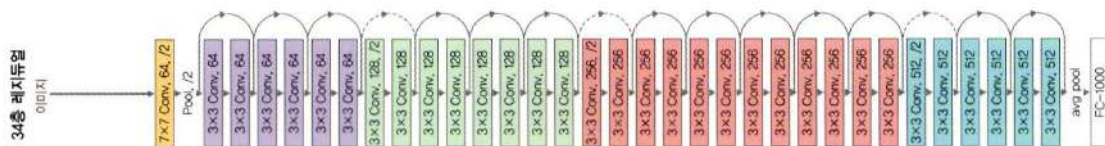
- 신경망이 깊어져도 일정 단계를 지나면 오히려 성능이 나빠짐
- 해결: residual block; 기울기가 잘 전파될 수 있도록 하는 shortcut(skip connection)

▼ 그림 6-25 ResNet 구조



- 블록: 합성곱층을 하나의 블록으로 묶음
- 묶인 계층 하나가 레지듀얼 블록(residual block)

▼ 그림 6-26 ResNet 모델 전체 네트워크



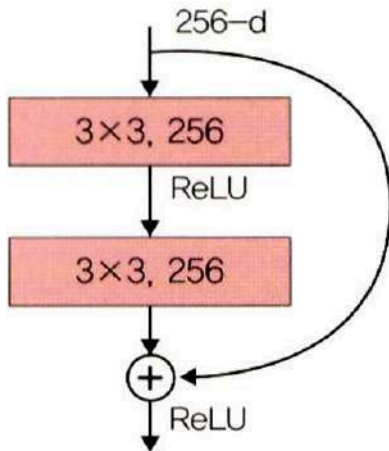
- **Stem:** 7×7 conv, stride 2 → 3×3 maxpool, stride 2 → (56, 56, 64)
- **Stage 1:** residual block × n₁ → (56, 56, C)
- **Stage 2:** residual block × n₂ (첫 블록에서 다운샘플) → (28, 28, 2C)
- **Stage 3:** residual block × n₃ (첫 블록에서 다운샘플) → (14, 14, 4C)
- **Stage 4:** residual block × n₄ (첫 블록에서 다운샘플) → (7, 7, 8C)
- **Head:** global average pooling → FC → 클래스 확률
- stage가 바뀔 때마다 가로 세로 절반, 채널은 두 배

모델	블록 종류	블록 수 (n ₁ , n ₂ , n ₃ , n ₄)
ResNet-18	Basic	2, 2, 2, 2
ResNet-34	Basic	3, 4, 6, 3
ResNet-50	Bottleneck	3, 4, 6, 3
ResNet-101	Bottleneck	3, 4, 23, 3
ResNet-152	Bottleneck	3, 8, 36, 3

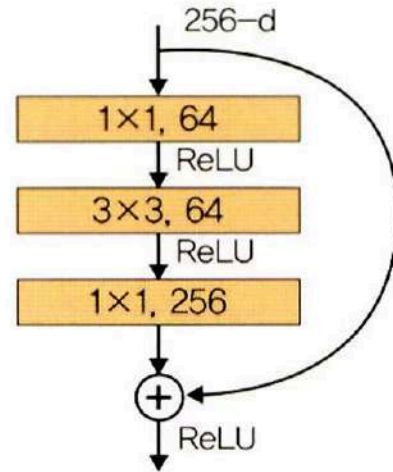
- 파라미터 수가 너무 커지는 문제 → 병목 블록(bottleneck)

▼ 그림 6-27 기본 블록과 병목 블록

**ResNet34
레지듀얼 블록**



**ResNet50
레지듀얼 블록**

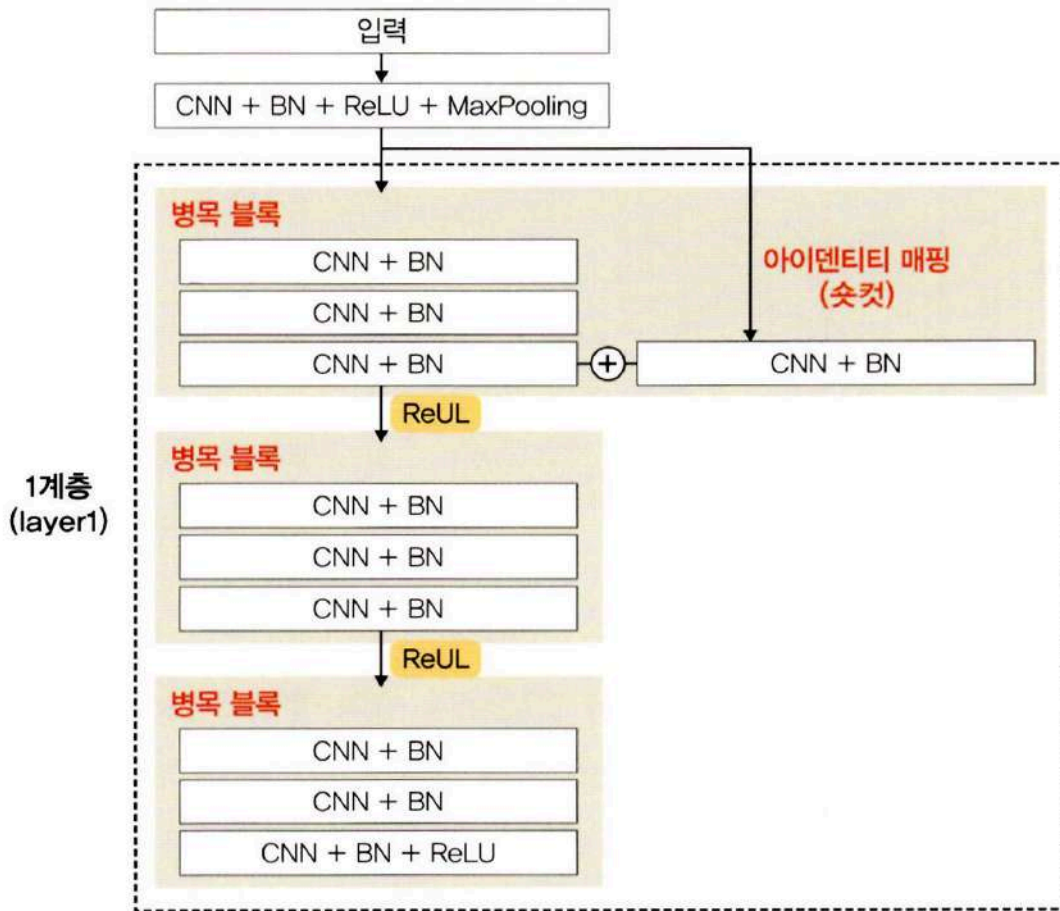


- 병목 블록을 사용해 파라미터 수를 감소할 수 있음: 3 x 3 합성곱층 앞뒤 1 x 1 합성곱층; 1 x 1 합성곱층이 채널의 수를 조절하며 차원을 줄였다가 늘리는 것이 가능 → 파라미터 수 줄일 수 있음
- 아이덴티티 매핑(identity mapping/ shortcut/ skip connection): 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태(shape)로 출력 되도록 함

ResNet에서 원래 입력을 더하는 이유

- 층이 너무 깊어지면 학습 데이터에서 오히려 성능이 떨어지는 현상 발생; 학습 자체가 잘 안됨
- 아무것도 안 하기가 쉬워짐
 - 어떤 층이 필요가 없다면 층은 입력을 그대로 내보내는(항등함수)를 배워야함.
 - 그러나 conv + ReLU가 항등함수를 학습하는 것은 어려움
 - 출력이 $F(x) + x$ 인 경우 $F(x)=0$ 만 만들면 출력이 x 가 됨
 - 가중치를 0 근처로 보내는 것은 쉬움
 - 따라서 층을 쌓아도 최소한 손해는 보지 않음
 - 층이 전체가 아닌 차이만 학습하면 됨
 - 원하는 출력을 $H(x)$ 라 하면 $F(x) = H(x) - x$; 입력에서 얼마나 고치면 되는지(residual)만 학습하면 됨
 - 기울기가 잘 전달됨
 - 역전파 때 출력 $F(x) + x$ 를 x 로 미분할 경우 $F'(x) + 1$ 됨.
 - +1이 있기 때문에 $F'(x)$ 가 아주 작아져도 기울기가 앞쪽 층까지 사라지지 않고 흘러감

▼ 그림 6-28 아이덴티티 매핑(샷컷)

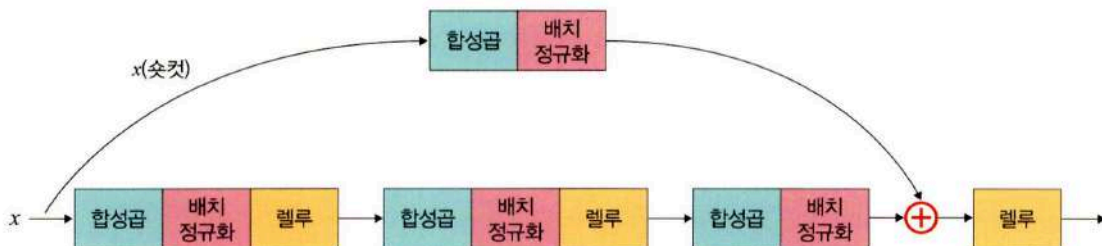


- bottleneck block
 - 층이 50개 이상 깊어지는 경우 3x3 conv 두 개는 계산량이 너무 커짐

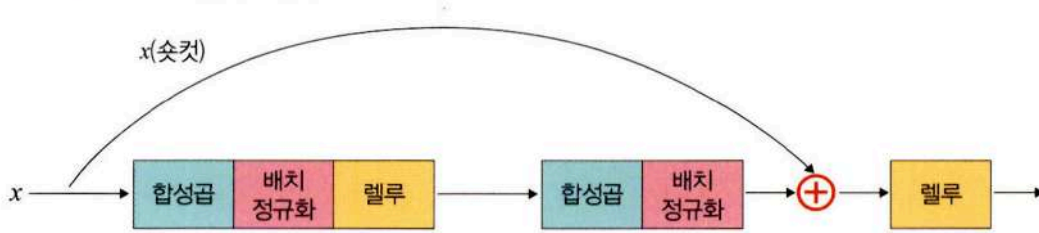
```

graph LR
    x["x (256ch)"] --> conv1["1x1 conv (-64) -> BN -> ReLU"]
    conv1 --> conv2["3x3 conv (64 -> 64) -> BN -> ReLU"]
    conv2 --> conv3["1x1 conv (-256) -> BN"]
    x -- shortcut --> add["(+)"]
    conv3 --> add
    add --> relu["ReLU"]
  
```

▼ 그림 6-30 합성곱 블록

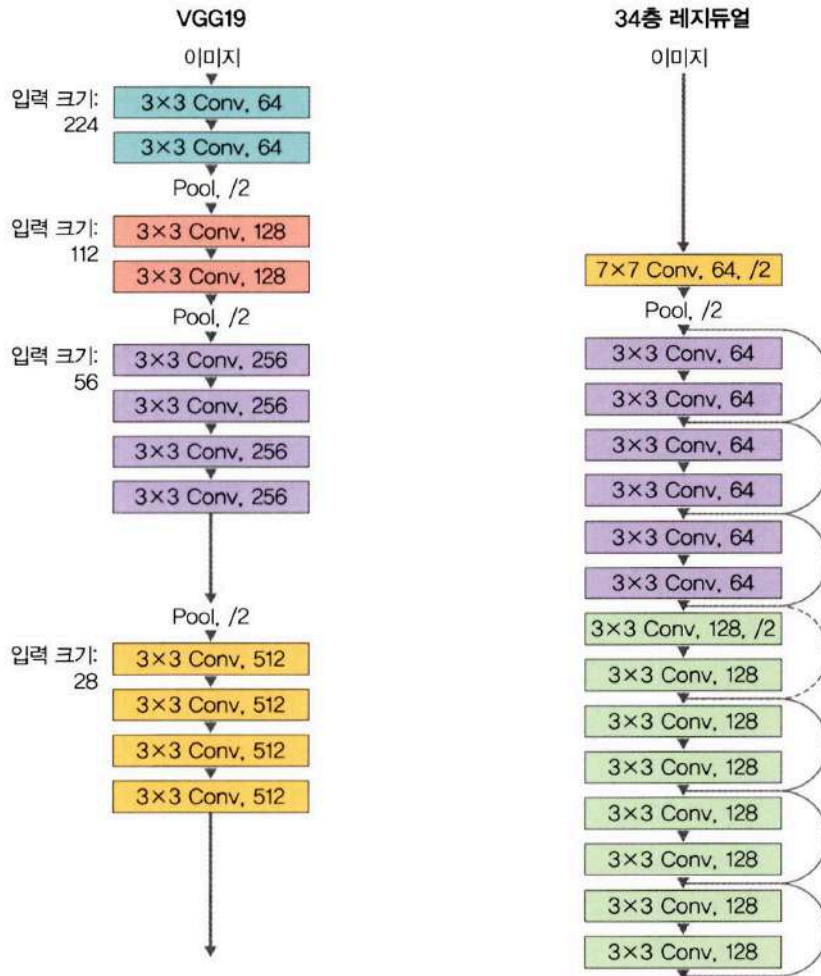


▼ 그림 6-31 아이덴티티 블록



- VGG19 구조 그대로, 합성곱 추가하여 깊게 만든 후 스킷 추가

▼ 그림 6-32 VGG19와 ResNet 비교



```
#6-65 필요한 라이브러리 호출
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np
```

```
import copy
from collections import namedtuple
import os
import random
import time

import cv2
from torch.utils.data import DataLoader, Dataset
from PIL import Image

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

- 네임드튜플(namedtuple): 튜플의 성질을 갖는 자료형; 인덱스뿐만 아니라 키 값으로 데이터 접근 가능

```
#6-66 이미지 데이터 전처리
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(resize),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }
    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

```
#6-67 변수에 대한 값 정의
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

```
#6-68 훈련과 데이터셋 불러오기
cat_directory = r'/content/drive/MyDrive/euron/4주차/080289-main/chap06/data/dogs-vs-cats/Cat'
dog_directory = r'/content/drive/MyDrive/euron/4주차/080289-main/chap06/data/dogs-vs-cats/Dog'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_directory)])

images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]

correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]
```

```
#6-69 불러온 데이터셋 훈련, 검증, 테스트 용도로 분리
random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

```
#6-70 이미지에 대한 레이블 구분
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase = 'train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path)
        img_transformed = self.transform(img, self.phase)
```

```

label = img_path.split('/')[-1].split('.')[0]
if label == 'dog':
    label = 1
elif label == 'cat':
    label = 0
return img_transformed, label

```

#6-71 이미지 데이터셋 정의

```

train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase = 'train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase = 'val')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])

```

#6-72 데이터셋의 데이터를 메모리로 불러오기

```

train_iterator = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_iterator = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_iterator, 'val': valid_iterator}

batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)

```

#6-73 기본 블록 정의

```

class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)

        if self.downsample is not None:
            i = self.downsample(i)

        x += i
        x = self.relu(x)

        return x

```

#6-74 병목 블록 정의

```

class Bottleneck(nn.Module):
    expansion = 4

    def __init__(self, in_channels, out_channels, stride = 1, downsample = False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, out_channels*self.expansion, kernel_size=1, stride=1, bias=False)
        # 다음 계층의 입력 채널 수와 일치하도록 self.expansion * out_channels
        self.bn3 = nn.BatchNorm2d(out_channels*self.expansion)
        self.relu = nn.ReLU(inplace=True)

```

```

if downsample:
    conv = nn.Conv2d(in_channels, out_channels*self.expansion, kernel_size=1, stride=stride, bias=False)
    bn = nn.BatchNorm2d(out_channels*self.expansion)
    downsample = nn.Sequential(conv, bn)
else:
    downsample = None
self.downsample = downsample

def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)
    x = self.relu(x)
    x = self.conv3(x)
    x = self.bn3(x)

    if self.downsample is not None:
        i = self.downsample(i)

    x += i
    x = self.relu(x)

    return x

```

#6-75 ResNet 모델 네트워크

```

class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()
        block, n_blocks, channels = config
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4

        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channels[1], stride=2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channels[2], stride=2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channels[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(self.in_channels, output_dim)

        if zero_init_residual:
            for m in self.modules():
                if isinstance(m, Bottleneck):
                    nn.init.constant_(m.bn3.weight, 0)
                elif isinstance(m, BasicBlock):
                    nn.init.constant_(m.bn2.weight, 0)

    def get_resnet_layer(self, block, n_blocks, channels, stride=1):
        layers = []
        if self.in_channels != block.expansion * channels:
            # in_channels와 block.expansion * channels가 다르면 downsample 적용
            downsample = True
        else:
            downsample = False
        layers.append(block(self.in_channels, channels, stride, downsample))
        self.in_channels = block.expansion * channels

        for i in range(1, n_blocks):
            layers.append(block(self.in_channels, channels))
            self.in_channels = block.expansion * channels
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)
        x = self.layer1(x)

```

```

x = self.layer2(x)
x = self.layer3(x)
x = self.layer4(x)
x = self.avgpool(x)
h = x.view(x.shape[0], -1)
x = self.fc(h)

return x, h

```

- if zero_init_residual
 - 각 레지듀얼 분기(residual branch)에 있는 마지막 BN(batch normalization)을 0으로 초기화해서 다음 레지듀얼 분기를 0에서 시작할 수 있도록 함

```

#6-76 ResNetConfig 정의
ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])

```

```

#6-77 기본 블록을 사용하여 ResNetConfig 정의
resnet18_config = ResNetConfig(block=BasicBlock, n_blocks=[2, 2, 2, 2], channels=[64, 128, 256, 512])
resnet34_config = ResNetConfig(block=BasicBlock, n_blocks=[3, 4, 6, 3], channels=[64, 128, 256, 512])

```

```

from torchvision.models.resnet import resnet152
#6-78 병목 블록을 사용하여 ResNetConfig 정의
resnet50_config = ResNetConfig(block=Bottleneck, n_blocks=[3,4,6,3], channels=[64, 128, 256, 512])
resnet101_config = ResNetConfig(block=Bottleneck, n_blocks=[3,4,23,3], channels=[64, 128, 256, 512])
resnet152_config = ResNetConfig(block=Bottleneck, n_blocks=[3,8,36,3], channels=[64, 128, 256, 512])

```

```

#6-79 사전 훈련된 ResNet 모델 사용
pretrained_model = models.resnet50(pretrained=True)

```

```

print(pretrained_model)

```

```

#6-81 ResNet50 Config를 사용한 ResNet 모델 사용
OUTPUT_DIM = 2
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)

```

```

#6-82 옵티마이저 손실 함수 정의
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

model = model.to(device)
criterion = criterion.to(device)

```

```

#6-83 모델 학습 정확도 측정 함수 정의
def calculate_topk_accuracy(y_pred, y, k=2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1)
        top_pred = top_pred.t()
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred))
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim=True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim=True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k

```

- , top_pred = y_pred.topk(k, 1)
 - tensor.topk 는 torch.argmax와 같은 효과
 - 주어진 텐서에서 가장 큰 값의 index를 얻기 위해 사용
- t()
 - 차원 0과 1을 전치(transpose)하겠다는 의미
- 텐서 비교 함수
 - torch.eq: 텐서가 서로 같은지 비교
 - torch.ne: 텐서가 다른지 비교
 - torch.ge: 크기가 같은지 비교
 - torch.eq(비교 대상 텐서, 비교할 텐서)

#6-84 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, scheduler, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0
    model.train()

    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item()
        epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)

    return epoch_loss, epoch_acc_1, epoch_acc_5
```

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred = model(x)
            loss = criterion(y_pred[0], y)

            acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
            epoch_loss += loss.item()
            epoch_acc_1 += acc_1.item()
            epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)

    return epoch_loss, epoch_acc_1, epoch_acc_5
```

#6-86 모델 학습 시간 측정 함수 정의

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time/60)
    elapsed_secs = int(elapsed_time-(elapsed_mins*60))
    return elapsed_mins, elapsed_secs
```

#8-87 모델 학습

```
best_valid_loss = float('inf')
EPOCHS = 10

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss, train_acc_1, train_acc_5 = train(model, train_iterator, optimizer, criterion, scheduler, device)
    valid_loss, valid_acc_1, valid_acc_5 = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'resnet50-dogcat.pt')

    end_time = time.monotonic()
```

```

epoch_mins, epoch_secs = epoch_time(start_time, end_time)

print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
print(f'\tTrain Loss: {train_loss:.3f} | Train Acc@1: {train_acc_1*100:6.2f}% | Train Acc@5: {train_acc_5*100:6.2f}%')
print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc@1: {valid_acc_1*100:6.2f}% | Valid Acc@5: {valid_acc_5*100:6.2f}%')

```

#6-88 테스트 데이터셋을 이용한 모델 예측

```

import pandas as pd
id_list = []
pred_list = []

_id = 0
with torch.no_grad():
    for test_path in test_images_filepaths:
        img = Image.open(test_path)
        _id = test_path.split('/')[1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=1)[0].tolist()
        id_list.append(_id)
        pred_list.append(preds[0])
res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)
res.to_csv('submission.csv', index=False)
res.head(10)

```

#6-89 모델 예측에 대한 결과 출력

```

class_ = classes = {0: 'cat', 1: 'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]

        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()

display_image_grid(test_images_filepaths)

```

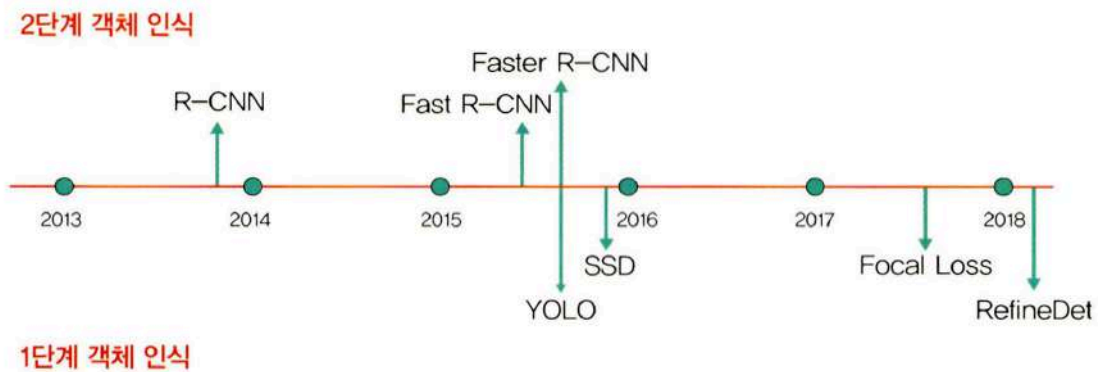


6.2 객체 인식을 위한 신경망

- 객체 인식: 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 학습
- 이미지나 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스(bounding box)로 나타내는 위치 검출(localization) 문제를 다룬

객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출

▼ 그림 6-35 1단계 객체 인식 vs 2단계 객체 인식 흐름도

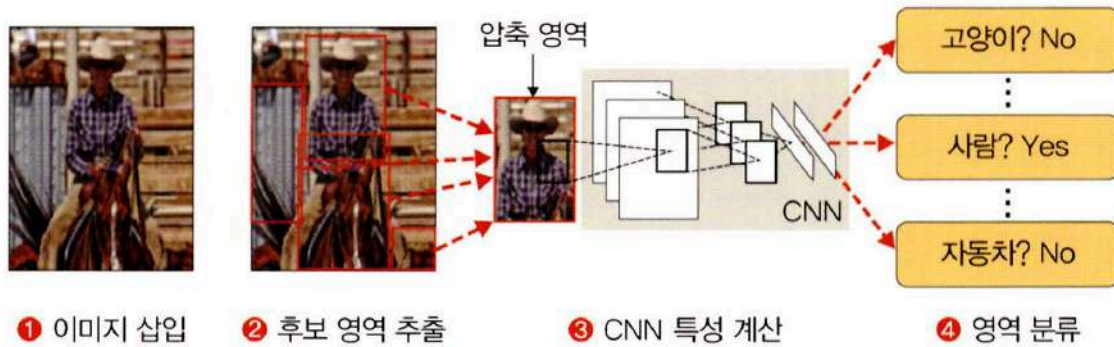


- 1단계 객체 인식
 - 분류와 위치 검출 동시에
 - 비교적 빠르지만 정확도 낮음
- 2단계 객체 인식
 - 분류와 위치 검출 순차적으로
 - 비교적 느리지만 정확도가 높음

6.2.1 R-CNN

- 선택적 탐색(selective search) 알고리즘을 적용한 후보 영역(region proposal) 많이 사용

♥ 그림 6-36 R-CNN 학습 절차



1. 이미지 입력으로 받음
2. 2000개의 bounding box를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고 CNN 모델에 넣기 위해 같은 크기(227 x 227 픽셀)로 통일(warping)
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델 적용
4. 각각 분류 진행해 결과 도출

선택적 탐색

객체 인식이나 검출을 위한 가능한 후보 영역을 알아내는 방법
분할 방식 이용해 시드(seed) 선정, 시드에 대한 완전 탐색 적용

1. 초기 영역 생성(sub-segmentation)
 - a. 각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역 생성
 - b. 입력된 이미지를 영역 다수 개로 분할

♥ 그림 6-37 R-CNN 학습 1단계



2. 작은 영역의 통합
 - a. 영역 여러 개로 나눈 것들을 비슷한 영역군으로 통합
 - b. 탐욕(greedy) 알고리즘 사용해 비슷한 영역이 하나로 통합될 때까지 반복

♥ 그림 6-38 R-CNN 학습 2단계



3. 후보 영역 생성

a. 2단계에서 통합된 이미지 기반으로 후보 영역(바운딩 박스) 추출

♥ 그림 6-39 R-CNN 학습 3단계



- 완전 탐색(exhaustive search): 후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려해 후보 영역을 찾는 기법
- 분할(segmentation): 영상 데이터의 특성에 따라 분할하여 후보 영역을 선정하는 기법
- 후보 영역(바운딩 박스): 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
- 시드(seed): 영상에서는 특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해나가며 분할함. 특정 기준점이 되는 픽셀이 시드

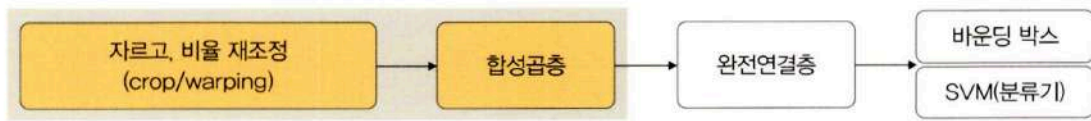
- R-CNN의 단점
 - 복잡한 세 단계의 학습 과정
 - 긴 학습 시간과 대용량 저장 공간
 - 객체 검출 속도 문제

6.2.2 공간 피라미드 풀링

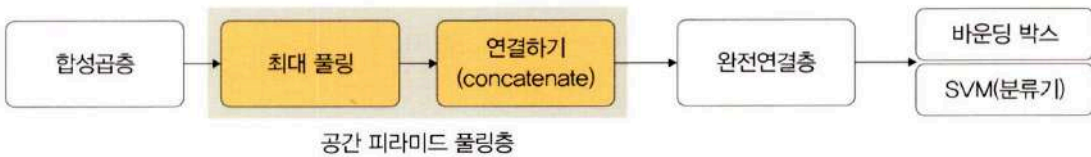
- 문제: 기존 CNN 구조는 모두 완전연결층을 위해 입력 이미지를 고정했어야함
- → 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나(crop) 비율을 조정(warp)해야했음
- 물체의 일부만이 잘리거나 본래의 생김새와 달라짐
- 해결: 공간 피라미드 풀링(spatial pyramid pooling)

▼ 그림 6-40 공간 피라미드 풀링

R-CNN



공간 피라미드 풀링

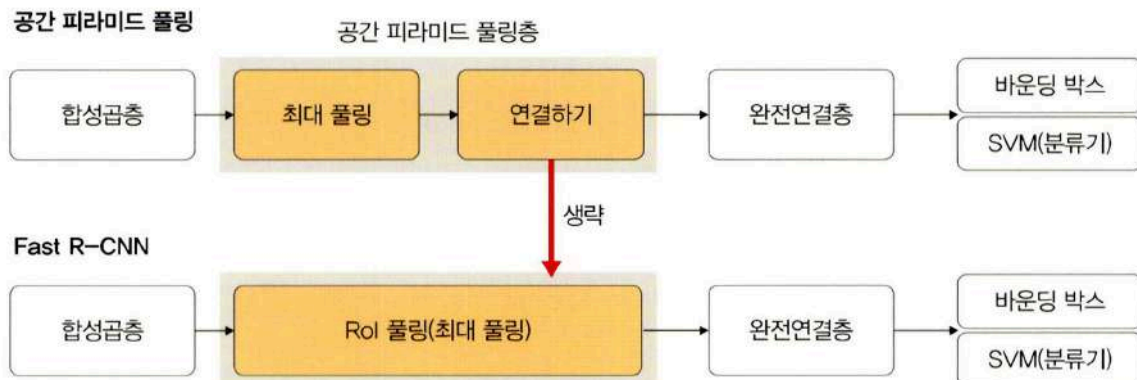


- 입력 이미지 크기에 관계 없이 합성곱층 통과
- 완전연결층에 전달되기 전 특성 맵들을 동일한 크기로 조절해주는 풀링층 적용
- 원본 이미지의 특징이 훼손되지 않는 특성 맵 얻을 수 있음

6.2.3 Fast R-CNN

- R-CNN 속도 문제 개선하고자 ROI 풀링 도입
- 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 함
- 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기 조정

▼ 그림 6-41 Fast R-CNN



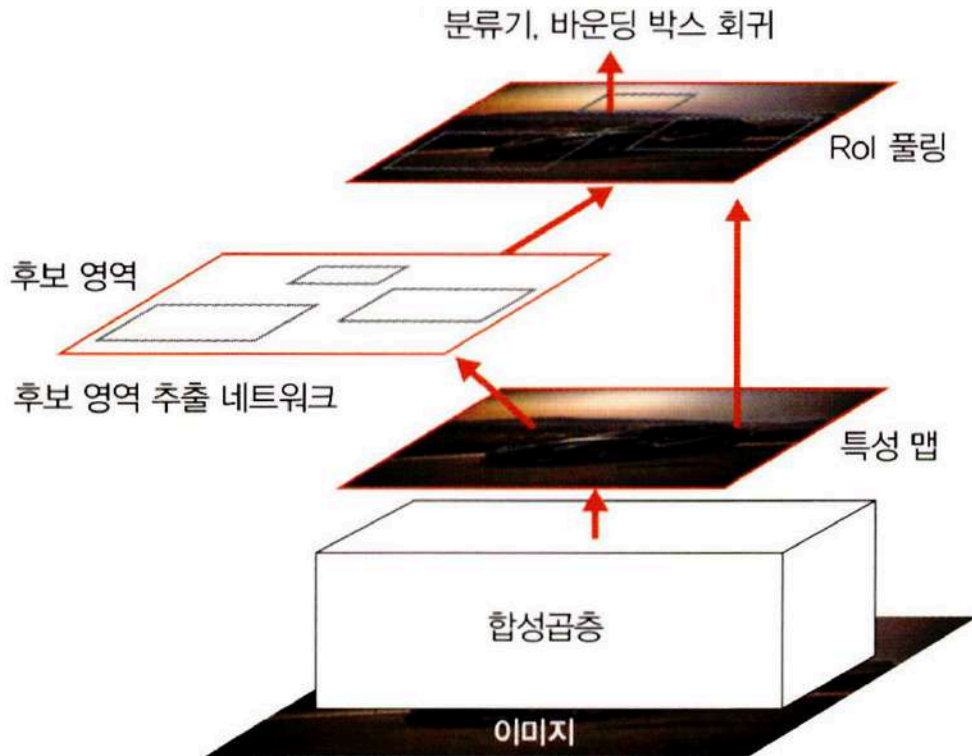
ROI 풀링

크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링 적용 → 결괏값 크기 동일하게 맞춤

6.2.4 Faster R-CNN

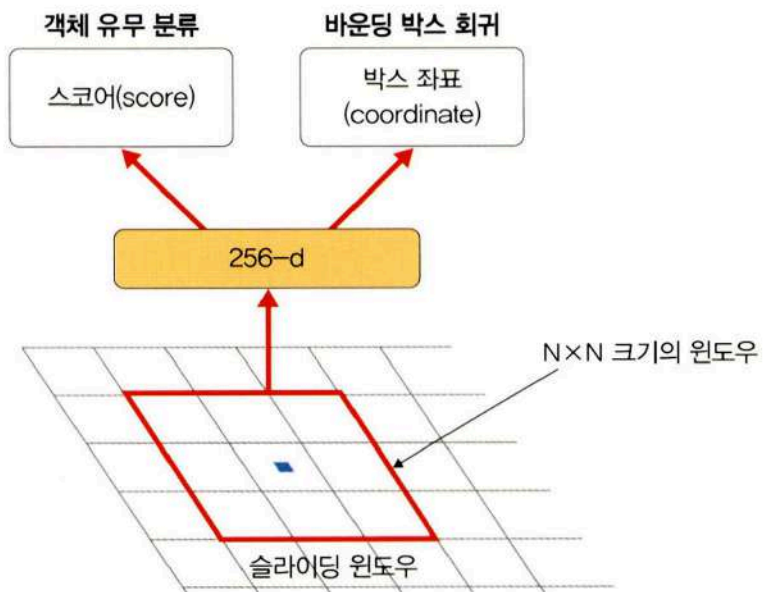
- 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계
- 기존 Fast R-CNN에 후보 영역 추출 네트워크(region proposal network RPN) 추가

♥ 그림 6-43 Faster R-CNN



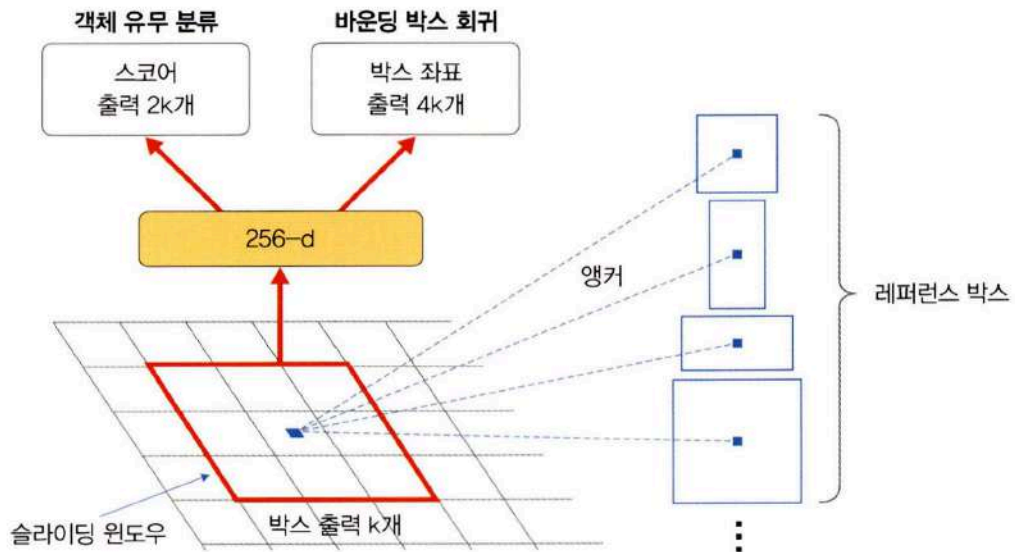
- 마지막 합성곱층 다음 위치
- RoI 풀링, 분류기(classifier), 바운딩 박스 회귀(bounding-box regression) 위치
- 해당 영역에 객체의 존재 유무 판단 위해 이진 분류(binary classification) 수행하는 작은 네트워크 생성

♥ 그림 6-44 후보 영역 추출 네트워크



- 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 $N \times N$ 크기의 입력만으로 다양한 크기와 비율의 이미지를 수용하기 어려움
- 여러 크기와 비율의 레퍼런스 박스 k 개 정의; 각각 슬라이딩 윈도우 위치마다 박스 k 개 출력하도록 설계(anchor)

♥ 그림 6-45 앵커

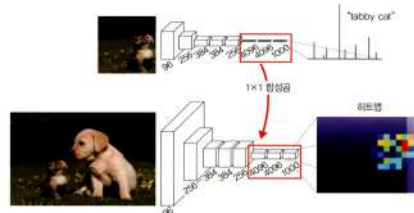


6.3 이미지 분할을 위한 신경망

6.3.1 완전 합성곱 네트워크

- 완전 연결층 한계: 고정된 크기의 입력만 받아들임
- 완전 연결층 거친 후 위치 정보가 사라짐
- 완전 연결층을 1x1 합성곱으로 대체

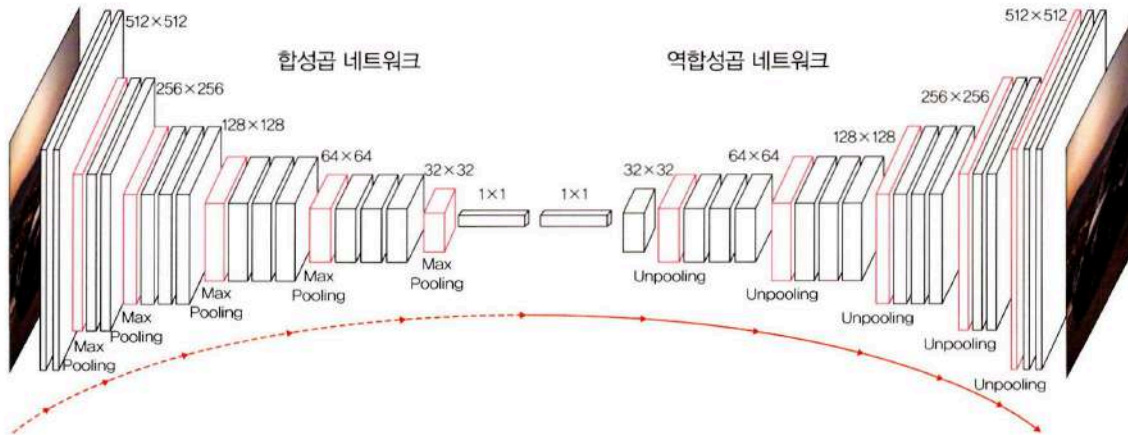
♥ 그림 6-46 완전 합성곱 네트워크



6.3.2 합성곱 & 역합성곱 네트워크

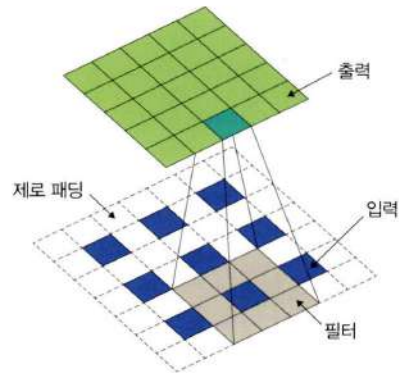
- 완전 합성곱 네트워크 단점
 - 여러 단계의 합성곱층과 풀링층을 거치며 해상도가 낮아짐
 - 낮아진 해상도를 복원하기 위해 업 샘플링 방식으로 사용 → 이미지 세부 정보 잃어버림
- 해결: 역합성곱 네트워크 도입: convolutional & deconvolutional network
- CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용
- semantic segmentation 등에 활용
- upsampling

▼ 그림 6-47 합성곱 & 역합성곱 네트워크



- 역합성곱 작동 방식
 - 각각의 픽셀 주위에 제로 패딩 추가
 - 패딩된 것에 합성곱 연산 수행

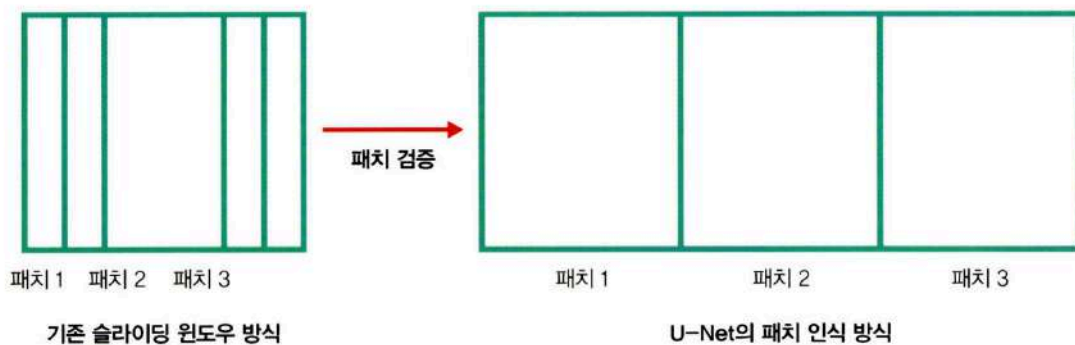
▼ 그림 6-48 역합성곱 진행 방식



6.3.3 U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망
- 특징
 - 속도가 빠름: 기존 슬라이딩 윈도우 방식은 이전 패치(patch)에서 검증이 끝난 부분을 다음 패치에서 또 검증 → 속도가 느림
 - U-Net은 이미 검증이 끝난 패치는 건너뛴 → 속도 빠름

▼ 그림 6-49 슬라이딩 윈도우 방식



- 트레이드 오프에 빠지지 않음: 패치 크기가 커지면 넓은 범위의 이미지 인식에 뛰어남 → 컨텍스트 인식에 탁월 but 지역화에는 한계
- 너무 넓은 범위 한 번에 인식, 지역화에는 약함

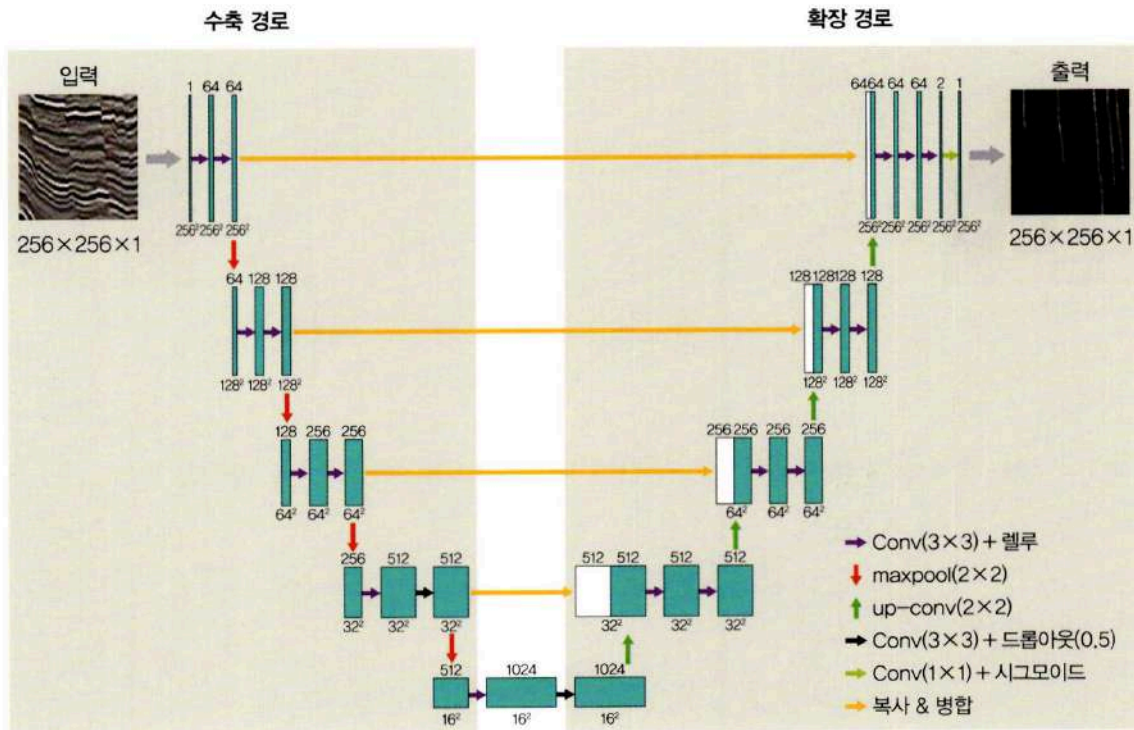
- U-Net은 컨텍스트 인식과 지역화 트레이드오프 문제 개선

지역화

- 이미지 안에 객체 위치 정보를 출력해줌
- 바운딩 박스 많이 사용
- 바운딩 박스 네 꼭지점 픽셀 좌표 출력이 아닌 왼쪽 위, 오른쪽 아래 좌표 출력

- U-Net 구조

♥ 그림 6-51 U-Net

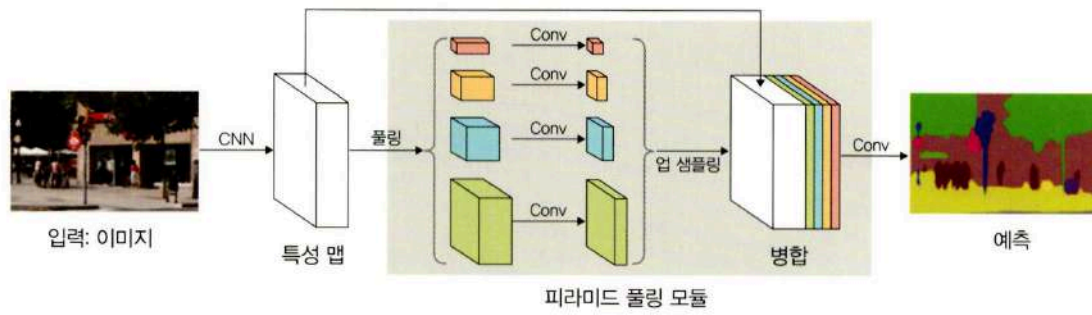


- FCN 기반; 수축 경로와 확장 경로
- 수축 경로: 컨텍스트 포착
- 확장 경로: 특성 맵 업 샘플링, 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합, 정확한 지역화 수행
- 각 합성곱 블록 3 x 3 합성곱 두 개로 구성, 그 사이 드롭아웃

6.3.4 PSPNet

- 완전연결층 한계 극복 위해 피라미드 풀링 모듈 추가
- 훈련 과정
 1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링
 2. 1×1 합성곱 사용하여 채널 수 조정, 풀링층 개수 N이라 할 때 출력 채널 수 = 입력채널 수 / N
 3. 모듈 입력 크기에 맞게 특성 맵 업 샘플링; 양선형 보간법 사용됨
 4. 원래 특성맵과 1~3 과정에서 생성한 새로운 특성 맵 병합

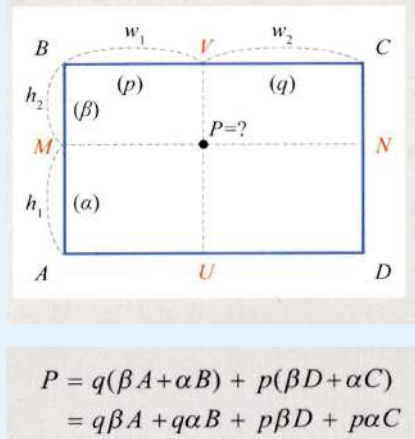
♥ 그림 6-52 PSPNet



양선형 보간법

- 보간법: 빈 화소에 값을 할당하여 좋은 품질의 영상 만들
- 선형 보간법: 원시 영상의 화소 값 두 개 사용; 원하는 좌표에서 새로운 화소 값을 계산
- 양선형 보간법: 화소당 선형 보간 세 번 수행, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치 곱한 값 합해서 얻음

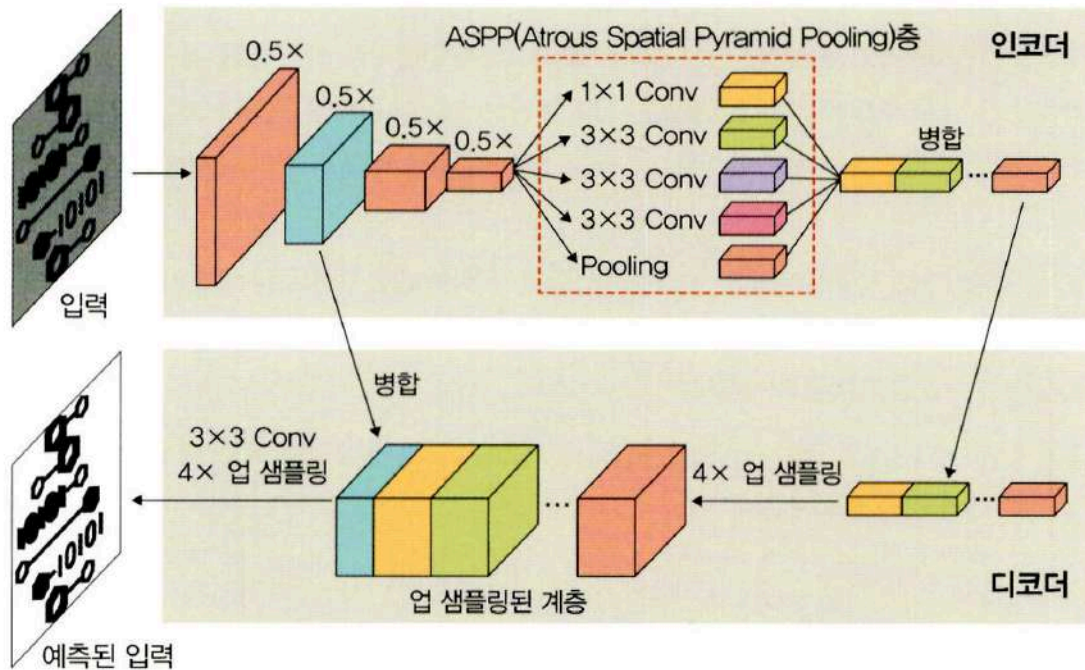
♥ 그림 6-53 양선형 보간법



6.3.5 DeepLabv3/DeepLabv3+

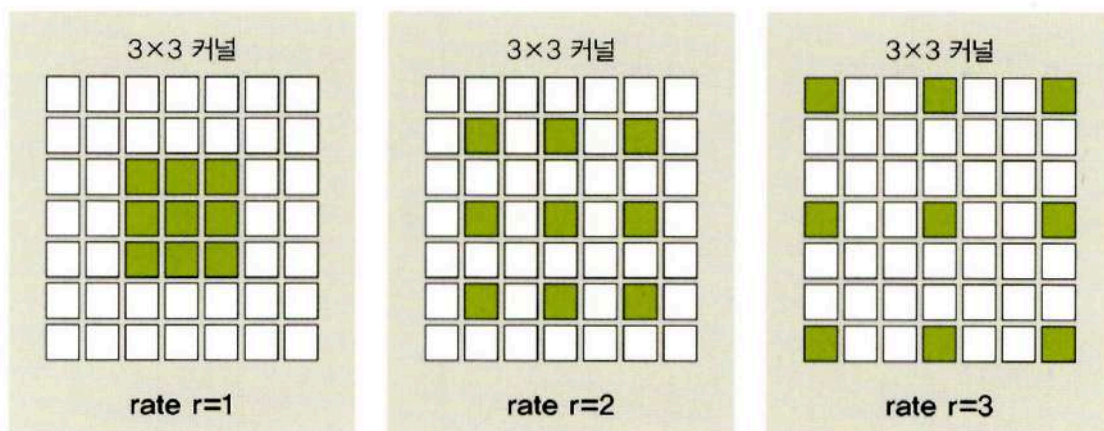
- 완전연결층 단점 보완 위해 Atrous 합성곱 사용
- 인코더와 디코더 구조

▼ 그림 6-54 DeepLab의 인코더-디코더 구조



- 얼마나 많은 빈 공간을 가질지 결정하는 파라미터 $rate$; $rate$ 이 커질수록 빈 공간 더 많아짐

▼ 그림 6-55 Atrous 합성곱



✔ 그림 6-56 Atrous 합성곱 효과

